

Learning from Click-Logs: a Two-Stage Retrieve-then-Rank System for MIND and EB-NeRD

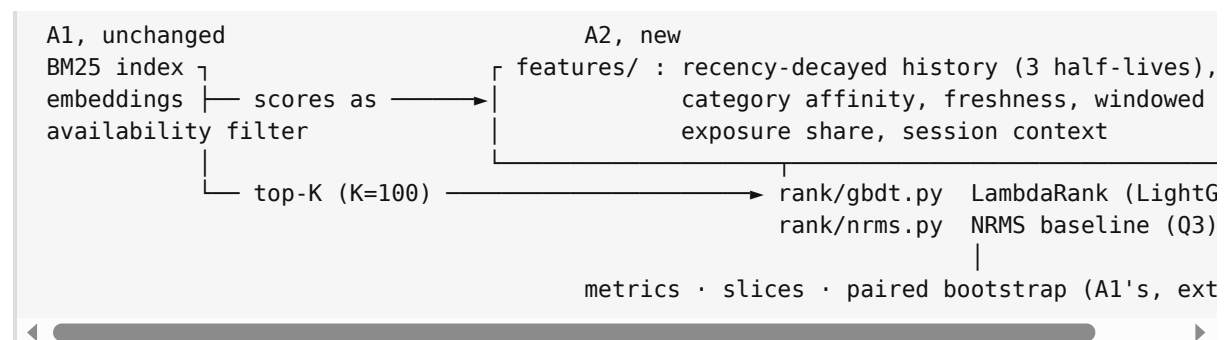
CS4.406 Information Retrieval & Extraction — Assignment 2 · Chaitanya Shah Done **solo**, which the specification permits for this team assignment.

Code: `src/newsrec/` · decision log: `ARCHITECTURE.md` (A1's D1–D32, A2's D33–D40) · every quoted figure traces to `reports/NUMBERS.md`.

1. What was built, and the one idea behind it

A1 ended on a measured gap rather than a result: **BM25 and embedding retrieval contain no time term** — a structural fact about the two formulas, not a tuning failure — while **92.7% (MIND) / 93.5% (EB-NeRD) of real clicks are on fresh articles**. Three independent routes reached that conclusion, the sharpest being that train-window popularity predicts EB-NeRD clicks in the *wrong* direction (AUC 0.4647), because 86.9% of validation candidates were never clicked during training at all.

A2 acts on it. The A1 scorers are not replaced; **they become features** of a trained re-ranker that also sees time:



Because `cos_inf` (infinite half-life) reproduces A1's semantic score exactly and `bm25` is A1's lexical score, **"before re-ranking" is one column of "after"** — the Q2 comparison is two orderings of the same candidate list, not two unrelated systems.

Two candidate sets, deliberately kept apart (D37). Both leaderboards ask us to re-rank a *supplied* list (`article_ids_inview` / `impressions`); Q2 also asks us to *retrieve* top-K from the whole corpus. These are different operations on different candidate sets, and conflating them is the easiest way to report a meaningless number. Every table below says which it is.

2. Q1 — features, and the boundary that makes them honest

Feature	MIND	EB-NeRD
<code>cos_short / cos_medium / cos_inf</code> — decayed user profile · candidate (D35)	✓ (3/10/∞ positions)	✓ (1 d / 7 d / ∞)
<code>cat_share_short / medium / inf</code> — recency-weighted category affinity	✓	✓
<code>bm25</code> — A1's lexical score, N=100 (D31)	✓	✓
<code>freshness_hours</code> — $T - \min(\text{published_time}, \text{first_seen})$ (D34c)	✓ (first_seen only)	✓
<code>exposure_share_1h / _24h</code> — label-free share, strictly before T (D34b)	✓	✓
<code>hours_since_last_click</code> (D35)	—	✓
<code>session_position</code> , <code>minutes_since_session_start</code> , <code>device_type</code> (D34a)	—	✓

The feature set is an **allowlist**, never a **denylist**, and the builder *asserts* its output columns equal keys + label + that list. This is not tidiness: `total_pageviews` and `total_inviews` already sit in our `articles.parquet` and are whole-dataset aggregates containing the future, so any builder that loops over article columns picks them up and nothing errors.

Boundary enforcement (Q1.4, Q9.2). `tests/test_no_leakage.py` asserts two *properties* on real validation data for both datasets, rather than a list of per-feature checks: *future deletion* (delete every impression after τ ; features at or before τ must be byte-identical) and *label blindness* (shuffle the clicks; every feature must be identical). A new feature is covered automatically. The suite is mutation-verified: four realistic forward leaks were planted in production code and **all four were caught** — but only after extending the check to MIND, because on EB-NeRD `published_time` almost always wins the `min`, so the `first_seen` path is never exercised there.

Three landmines the data set for us, each measured rather than assumed. EB-NeRD's article catalogue runs to 2023-07-11, a month past the test week (2,148 articles are published after the train week ends), so **no catalogue-wide statistic is ever a feature**. EB-NeRD's `session_id` is shared by more than one user (1,032 ids), so sessions are keyed **(user, session)**: keyed by id alone the longest "session" is 14 days, keyed properly it is 29 minutes. And 1,810 candidate rows (11 articles) are shown *before* their `published_time`, which is why freshness takes the earliest evidence of existence.

3. Q2 — the trained re-ranker, before and after

Model (D36). LightGBM with the `lambdarank` objective: it is scored only on ordering each impression's clicked candidates above its unclicked ones, weighted by how much a swap would move nDCG. Trained on the train split, tree count chosen by validation nDCG@10, **every number below from the local test split**, which chose nothing. Hyperparameters were fixed before any result was seen. LightGBM learns impression boundaries from group *sizes* alone, so a shuffle trains happily and ranks garbage; `group_sizes` derives them from the ids and refuses non-contiguous input.

3.1 Supplied inview lists — what the leaderboards grade

System	MIND AUC	MIND nDCG@10	EB-NeRD AUC	EB-NeRD nDCG@10
random	0.4982	0.2756	0.5007	0.4288
bm25 (A1)	0.5641	0.3265	0.5041	0.4305
semantic (A1) — <i>before</i>	0.6097	0.3588	0.5457	0.4585
LambdaRank (A2) — <i>after</i>	0.6144	0.3636	0.7428	0.6250
paired gain over <i>before</i>	+0.0048 [0.0012, 0.0080]	+0.0048	+0.1971 [0.1953, 0.1989]	+0.1665

Both AUC gains exclude zero. **The asymmetry is the finding, not a disappointment.** Feature importances say why: on EB-NeRD, freshness (38%) and exposure share (21% + 13%) carry 73% of the model's gain; on MIND, `cos_inf` leads at 27% and freshness at 20%, and the model saturates after 18 trees — validation nDCG@10 is flat from ~20 to 300 trees while training nDCG@10 climbs 0.396 → 0.457. Single-feature AUCs on MIND put freshness at 0.534 and exposure at 0.533 against similarity's 0.610. **MIND ships no publication times and no history timestamps**, so its freshness must be inferred from first appearance in a sampled log, and its recency can only use list position. The time signal helps where the data records time and cannot help where it does not; that is a property of the datasets that the design had to absorb, and it is why the two datasets' decay features are not comparable to each other.

3.2 Retrieved top-K (K = 100) — our own candidate generation

	MIND	EB-NeRD
recall@100, in-circulation pool	2.6%	2.5%
impressions with ≥1 click retrieved	798 of 21,289	4,704 of 186,721
MRR, retrieval order → re-ranked	0.070 → 0.260	0.052 → 0.567
AUC, retrieval order → re-ranked	0.567 → 0.786	0.508 → 0.968

Two honest readings. First, **recall@K is the ceiling**: re-ranking cannot rescue the 97.5% of impressions whose clicked article never entered the top 100, which is why both leaderboards score a supplied list instead. Second, **EB-NeRD's 0.968 is not a better system than §3.1's 0.743**: the 99 negatives here were drawn from a 125,541-article corpus by text similarity, so most were not in circulation that day, and freshness separates them almost trivially. Easy negatives inflate AUC. The supplied-list number is the one to believe.

4. Q3 — baseline reproduced, then beaten

Baseline (D39): NRMS (Wu et al., 2019) — multi-head self-attention over the user's clicked articles, then additive attention to pool them, scored by dot product against the candidate, trained with one click against four same-impression negatives. **One documented deviation**: the news encoder is a learned projection of our existing 384-dim multilingual sentence embeddings instead of word-level attention over title tokens. That is what makes it trainable on a GPU-less laptop; the user encoder, objective and scoring are the paper's.

The one principled change (D40): a time term. NRMS has no time input at all. A small MLP over (log freshness, exposure share 1 h, exposure share 24 h) is added to the click score, zero-initialised so training *starts* at exactly the baseline. The change was chosen from A1's measurements **before** any NRMS result existed — A1's Finding 5 is that selecting a claim after seeing its result is a tautology in disguise.

5. Q4 — serving: memory, p99, and cost per 1,000 queries

Measured per **single user request** ($K = 100$), single process, **with nothing else running** — NRMS training was suspended for the measurement, because A1's error log already contains a conclusion that had to be withdrawn after being taken on a contended machine.

	MIND	EB-NeRD
resident footprint, total	283 MB	475 MB
— largest items	embeddings 100 MB · user vectors 77 MB · exposure index 71 MB · BM25 19 MB	embeddings 193 MB · exposure index 123 MB · user vectors 61 MB · session table 44 MB
start-up (context build)	5.2 s	137 s at leaderboard scale
stage 1, retrieve top-100	p50 9.0 ms , p99 23.7 ms	p50 9.6 ms , p99 22.2 ms
stage 2, build features	p50 223.0 ms , p99 348.0 ms	p50 439.0 ms , p99 2,352 ms
stage 3, score 100 candidates	p50 3.2 ms , p99 9.0 ms	p50 4.3 ms , p99 6.9 ms
total	p50 237 ms, p99 360 ms	p50 452 ms, p99 2,369 ms
throughput per core	4.2 q/s	1.5 q/s
cost per 1,000 queries (at \$0.04 / vCPU-hour)	\$0.0027	\$0.0077
cores for 1,000 QPS	239	689
p99 < 100 ms SLA	not met	not met

Little's law as a consistency check. With one request in flight, $X \leq 1/R$: MIND's $R = 0.237$ s gives $X \leq 4.2$ q/s, which is exactly the measured per-core throughput. The model is therefore internally consistent, and the only way past 4.2 q/s per core is to cut R or add concurrency.

We fail the SLA, and the reason is architectural rather than algorithmic. Stage 1 (9 ms) and stage 3 (3–4 ms) are comfortably inside budget. **Feature assembly is 93-97% of the request**, and the honest diagnosis is that we are calling a *batch* builder once per request:

- Only **13.5% (MIND) / 16.8% (EB-NeRD)** of that stage is per-user profile work — three decayed profiles, three category-share tables, the BM25 query — measured separately. So "precompute user profiles nightly" is necessary but **not** sufficient; it buys ~30 ms of 223 ms.
- The remainder is fixed per-call cost: DataFrame joins against whole-dataset tables (a 1.2M-row session table, 125k articles, a 65k-entry category map) to answer a 100-row question. The same code processes **~11,500 rows/s in**

bulk (2.25M rows in 195 s) and **~450 rows/s one request at a time** — a $\sim 26\times$ penalty that is all overhead.

What it would take to meet $p99 < 100$ ms (a projection from the measurements above, not a measurement): precompute user profiles into a key-value store (~ 30 ms), replace the per-request joins with array lookups keyed by article row (the join targets are all static within an hour), and cache article-level features by (article, hour) — the same structure §8 wants at $10\times$. Stages 1 and 3 already sum to ~ 13 ms, so a feature lookup path of ~ 20 ms would put p99 near 40 ms and the cost per 1,000 queries near \$0.0003. **We did not build that**, and saying so is more useful than quoting a number we did not measure.

Where the money actually goes at our scale. The leaderboard run is the batch case and it is cheap: 206M candidate rows featurised and scored in 155 min on one machine, $\sim 22,000$ rows/s, i.e. the whole 13.5M-impression test set for well under a dollar of compute. Our system is a strong batch scorer and a weak online one, which is the correct thing to know about it.

6. Q5 — extended evaluation, sliced

All seven metrics, over the supplied-inview pipeline on the local test split, with 1,000-resample percentile CIs. Slices are D26's: cold = history ≤ 5 articles; head/tail by exposure, where an impression is head only if *every* clicked article is in the head set (2,922 MIND / 243 EB-NeRD impressions are mixed and counted in neither, rather than silently assigned).

	MIND: semantic (A1) → LambdaRank	EB-NeRD: semantic (A1) → LambdaRank
all	0.6097 → 0.6144, paired +0.0048 [0.0012, 0.0080]	0.5457 → 0.7428, +0.1971 [0.1953, 0.1989]
cold (n = 3,904 / 680)	0.5642 → 0.5529, -0.0113	0.5538 → 0.7348, +0.1810
warm (n = 18,043 / 186,041)	0.6195 → 0.6277, +0.0083	0.5457 → 0.7428, +0.1971
head-exposure (n = 13,108 / 98,719)	0.5955 → 0.5828, -0.0127 [-0.0176, -0.0079]	0.5565 → 0.7745, +0.2179
tail-exposure (n = 5,917 / 87,759)	0.6319 → 0.6671, +0.0353	0.5334 → 0.7076, +0.1741
diversity (category, K=10)	0.8166 → 0.7352	0.7993 → 0.8029
novelty (bits)	17.34 → 17.69	18.87 → 19.06
coverage	0.0283 → 0.0318	0.0260 → 0.0247

Three readings we would defend:

1. **On EB-NeRD the gain is not an artefact of one population** — it holds in every slice, largest on head-exposure (+0.218) and smallest on tail (+0.174). A model leaning on popularity should be expected to do *better* where articles are widely shown, and it does, which is a consistency check rather than a surprise.
2. **On MIND the aggregate +0.0048 hides two sign flips, which is exactly what slicing is for.** The model is *worse* on cold users (-0.0113) and *worse* on head-exposure impressions ($-0.0127 [-0.0176, -0.0079]$), and better on warm (+0.0083) and much better on tail-exposure (+0.0353). The mechanism

is measurable rather than guessed: on MIND a candidate's own exposure share predicts clicks in the **inverse** direction within its own rack (AUC 0.476 taken positively, 0.524 inverted), so the model learns to down-weight heavily-shown articles. That helps where the clicked article is obscure and hurts where it is the popular one. A cold user additionally has no decayed profile at all, so nothing compensates. Together these argue for a cold-start fallback and for rack-relative rather than absolute popularity features — the improvement we would make next, and the one §4's GBDT property predicts.

3. **The re-ranker is not free on diversity.** On MIND it trades category diversity (0.817 → 0.735) for novelty and coverage; on EB-NeRD it costs coverage (0.0260 → 0.0247) while raising novelty. Coverage is reported **without** a CI (D27): the bootstrap is biased downward for a union statistic, and the raw spread is carried in the CSV under its own column names instead.

7. Q9 — anti-gaming: leakage test, and the price of a serving-time feature

The test (Q9.2). Two properties, not a checklist: future deletion and label blindness (§2), mutation-verified against four planted leaks. A quarantined future feature *fails* the same check, which is how we know the check can see a leak at all.

The ablation (Q9.1). The shipped model is trained twice: on the allowlist, and on the allowlist plus everything in `features/unavailable.py` (`read_time`, `scroll_percentage`, `total_pageviews`, `future_exposure_share_24h`).

MIND (test, 21,947)	AUC	MRR
honest — this is what ships	0.6144 [0.6104, 0.6185]	0.3191
+ serving-unavailable	0.5994 [0.5952, 0.6033]	0.3014
paired difference	-0.0150 [-0.0181, -0.0118]	-0.0177

EB-NeRD (test, 186,721)	AUC	MRR
honest — this is what ships, and what both leaderboard files came from	0.7428 [0.7415, 0.7441]	0.5219
+ serving-unavailable	0.7590 [0.7578, 0.7602]	0.5487
paired difference	+0.0162 [0.0153, 0.0172]	+0.0269

The price of cheating on EB-NeRD is +0.016 AUC: real, but under a tenth of what the honest behavioural features bought (+0.197). The two columns that carry it are `total_pageviews` (17% of the leaky model's gain) and `future_exposure_share_24h` (16%).

`read_time` and `scroll_percentage` — the two features the schema most invites you to misuse — turn out to be inert here, and the reason is structural. EB-NeRD records dwell and scroll **per impression**, not per candidate, so they take the same value for every candidate in a rack. A ranking objective only ever compares candidates *within* a rack, so a rack-constant feature cannot change any ordering; neither reaches the leaky model's top five. That is worth stating because it cuts against the intuition that the most obviously post-hoc field is the

most dangerous one: **the dangerous leak is the one that varies per candidate**, which is why `total_pageviews` (a per-article whole-dataset aggregate) is the biggest offender in this table.

Cheating made MIND worse, and the reason is mechanical, not moral.

MIND ships no dwell or pageview fields, so the only quarantined column with data is the forward exposure window — and MIND's log ends 2019-11-15 23:58, so that window is 37–138 h wide in training but only 5.7–11.4 h at test time, inflating the feature's mean $3.2\times$ ($0.075 \rightarrow 0.241$). The leaky model leaned on it (118 trees against 18; 15.7% of its gain) and was then handed a differently-distributed input. **So MIND cannot price the leak**; EB-NeRD, where all four columns exist and three are not window-dependent, is where that price is measurable.

8. Where it breaks at 10×

The honest basis for this section is that we **already ran the largest thing we have**: 205,925,868 candidate rows (EB-NeRD) and 93,115,001 (MIND) were featurised and scored locally, in **155 min** and **26 min**, at a 10.6 GB peak on an 11 GB machine. So what follows extrapolates from a measured run rather than from an estimate.

Three things that made the current scale run, each a lesson about the next one.

1. *Never materialise the exploded candidate list as strings.* 206M (impression, article) pairs cost ~ 10 GB as Polars strings and 1.4 GB as one sorted `int64` key per pair; `searchsorted` over that array answers the popularity question in 40 ms per 59k rows.
2. *A deep slice of a lazy frame is not free once a row index is attached.* One 5,000-row chunk at offset 5M cost **297 s**, because the row index forces a scan of everything before it. Writing the prepared frame to Parquet first made it instant — the same class of error as assuming an operation is cheap because a *similar* one measured cheap.
3. *Partition by user, not by file order.* Per-user work (three decayed profiles, category shares, the BM25 query) costs ~ 3.4 ms and was being repeated in every chunk a user appeared in. Hashing users into 64 groups pays it once: 2.7 min per 215k-impression group.

At 10× (135M impressions, ~2 G candidate rows), in the order things break:

- **The exposure index, first.** 2 G `int64` keys is **16 GB** — it stops fitting. It must become either time-bucketed shards or a count keyed by (article, hour) instead of by impression, which is a bounded-size structure: 125k articles $\times$ 168 hours is 21M cells, not 2 G.
- **Per-user feature building, second.** It is linear and precomputed nowhere: ~ 8 M users $\times$ 3.4 ms $\approx$ **7.5 h** of profile building alone. A production system would move it to a nightly batch, because a user profile is a function of history, not of the request.
- **Wall clock, third.** ~ 26 h on this machine, which is past any deadline — but the work is embarrassingly parallel by user group, so it is a horizontal-scaling

problem rather than an algorithmic one. Nothing in the pipeline requires a global shuffle.

- **What does *not* break:** the model (322 trees, 1.6 MB), the embeddings (190 MB), and BM25 (a fixed sparse matrix). Stage 1 and stage 2 both stay within a single machine's memory at 10×; it is the *log-derived* features that grow with traffic.

One caveat about numbers taken under pressure. Profiling one leaderboard chunk showed `searchsorted` at 0.7 s per call; the same call on a quiet machine takes **10 ms**. The difference was memory pressure, not the array. A1's error log already contains a conclusion that had to be withdrawn for exactly this reason, which is why the latency figures in §5 were taken with nothing else running.

9. Reproducing, and what is ours

`README.md` carries the one-command rebuild. `AI_USAGE.md` marks every file as AI-generated, AI-generated-then-edited or human-written, and `reports/ai_transcripts/` holds the prompt log (Q7.4). The Codabench worker tooling under `codabench/` is infrastructure, not pipeline: the organisers' scoring program, hidden reference data and metrics are used unmodified.